

How To Build, Train, Test, and Deploy an AI Image Model

How To Build, Train, Test, and Deploy an AI Image Model Like This RC Car Project

This guide teaches the full workflow used in this project so another person could repeat it for their own image model from beginning to end.

This is not meant to be abstract theory only. It is meant to show the practical pipeline:

- collect your own camera images
- label them in Roboflow
- export the dataset
- train a YOLO segmentation model starting from `yolo26\*.pt`
- compare checkpoints
- test on real images
- test on live camera preview
- move the best model into your controller code

The example problem here is sidewalk segmentation for an RC car, but the workflow also applies to other vision problems.

1. Understand The Exact Goal

Before touching any training code, define the problem clearly.

In this project the task is:

- input: camera image from the RC car
- output: a mask for the sidewalk
- downstream use: autonomous steering

That means this is not simple image classification and not just bounding-box detection. It is segmentation, because the car cares about shape, edges, width, and center of the sidewalk.

Why this matters:

- a class label only tells you if sidewalk exists
- a box only tells you where it roughly is
- a mask gives you the actual geometry needed for steering

2. Use Your Real Camera, Not Random Internet Images

If you want a model that works on your RC car, use images from your RC car.

In this kind of project, the data should come from:

- the same camera hardware
- the same mounting angle
- the same field of view
- the same lighting conditions you expect during driving
- the same kinds of sidewalks and obstacles

Useful variation to collect:

- straight sidewalks
- left and right turns
- driveways
- curb cuts
- grass edges
- shadows

- cracked sidewalks
- different pavement colors
- bright sun and darker conditions
- partial sidewalk visibility

Bad data collection pattern:

- hundreds of near-identical frames from one easy route

Good data collection pattern:

- diverse routes, different angles, difficult cases, and scenes where the model is likely to fail

3. Label In Roboflow

For this workflow, use Roboflow for labeling.

Why Roboflow is a good fit:

- it is fast for image annotation
- it supports polygon and segmentation workflows
- it makes dataset review easier
- it exports in formats that are easy to convert or use

Recommended labeling process:

1. create a project in Roboflow
2. upload your RC car images
3. create one class if you are only segmenting sidewalk:
 - `sidewalk`
4. annotate masks carefully with polygons
5. review labels for consistency
6. fix mistakes before export

Important labeling rules:

- keep the mask tight to the visible sidewalk
- do not let the mask drift into grass unless that is intentional
- decide how to handle driveways and curb ramps consistently
- use the same rule on every image
- fix bad polygons instead of assuming the model will ignore them

If two similar images are labeled differently, the model learns confusion, not robustness.

4. Export From Roboflow

In this project, the manual sidewalk dataset is used as a COCO segmentation export and then converted into YOLO segmentation format for training.

That means a practical export path is:

- label in Roboflow
- export as COCO segmentation
- place the export into your project folder
- convert it into the YOLO segmentation dataset layout

In this repo, those dataset folders look like:

- `code/ai\_models/manual\_sidewalk\_detection1080p.coco-segmentation`
- `code/ai\_models/manual\_sidewalk\_detection240p.coco-segmentation`
- `code/ai\_models/manual\_sidewalk\_detection\_mixed.coco-segmentation`

The training script:

```
[train_manual_sidewalk_model.py] (/Users/ramsabavat/Desktop/rc_car_code/code/ai_models/train_manual_sidewalk_mo
```

does this conversion step automatically:

- reads Roboflow-style COCO annotations
- creates YOLO segmentation label files
- creates a fresh train/validation split
- writes a `data.yaml`

So the high-level data path is:

- Roboflow labeling
- COCO segmentation export
- import and remap into YOLO segmentation format
- train with Ultralytics

5. Choose The Base Model: Start From `yolo26\*.pt`

For this workflow, the correct baseline idea is not "train from scratch."

Start from a pretrained YOLO segmentation checkpoint such as:

- `yolo26s-seg.pt`

This is the exact kind of starting point used in the project experiments.

Why:

- small custom datasets usually are not enough to train strong segmentation from scratch
- pretrained checkpoints already understand edges, shapes, textures, and general object structure
- fine-tuning converges faster and usually generalizes better

Then, after you have a strong run, you can warm-start from your own previous checkpoint:

- `train14/best.pt`
- `train19/best.pt`
- or another proven run

So the model progression is usually:

1. start from `yolo26\*.pt`
2. train your first domain model
3. use your best previous checkpoint for later refinement

6. Build A Clean Training Script

Your training script should do the following:

- point to the exported dataset folder
- import and convert labels
- choose the model checkpoint
- define training settings
- train
- save the best checkpoint

In this repo, that script is:

```
[train_manual_sidewalk_model.py] (/Users/ramsabavat/Desktop/rc_car_code/code/ai_models/train_manual_sidewalk_mo
```

Core settings in this kind of run include:

- model checkpoint
- epochs
- image size
- batch size
- patience
- learning rate
- cache mode
- augmentation settings

The important experiment rule is:

- change one variable at a time

That is how you actually learn what helped.

7. Train Conservatively At First

A strong starting strategy for a small custom dataset is:

- segmentation checkpoint based on `yolo26\*.pt`
- moderate image size
- moderate batch size
- conservative learning rate
- limited augmentation

Why conservative settings matter:

- small datasets overfit easily
- changing too many hyperparameters at once makes results impossible to interpret
- a model that looks "more advanced" on paper can easily perform worse in practice

In this project, one of the biggest lessons was:

- warm starts from the right checkpoint mattered a lot
- changing many settings together often made the model worse

8. Track Every Run

Every training run should produce:

- a folder like `train15`, `train19`, `train24`, and so on
- an `args.yaml`
- a `results.csv`
- a `weights/best.pt`

The critical file is:

- `best.pt`

That is the checkpoint you should compare and deploy, not just the final epoch.

Things to compare from `results.csv`:

- best box `mAP50-95`
- best mask `mAP50-95`
- best epoch
- whether training plateaued early

But do not stop at mAP.

9. Compare On Real Images, Not Only Validation Metrics

This project showed an important real-world lesson:

- a model can win on validation mAP
- and still be worse on unseen driving images

So after training, you should test on:

- held-out route images
- difficult benchmark images
- live camera captures from the actual vehicle

In this repo, real-image evaluation was done on sets like:

- TPRF images
- 20-photo route samples

That is the right idea.

Because the real question is not:

- which model has the prettiest CSV

The real question is:

- which model actually sees the sidewalk correctly on new driving scenes

10. How To Test A Model On Saved Images

There are two good test styles.

Style A: Offline image inference with saved prediction folders

You can run a model against a folder or list of images and save outputs like:

- `train24\_tprf\_eval`
- `train24\_photo20\_eval`

This is useful for:

- side-by-side checkpoint comparison
- checking which images got masks
- visual inspection of boundary quality

Style B: Run the controller-side vision code on one image

There is also a local test utility:

[sidewalk_image_test.py] (/Users/ramsabavat/Desktop/rc_car_code/code/test_files/sidewalk_image_test.py)

That is useful for:

- taking one image
- running the current perception logic
- saving an annotated result
- writing raw analysis JSON

Typical use:

```
```bash
python3 /Users/ramsabavat/Desktop/rc_car_code/code/test_files/sidewalk_image_test.py
/path/to/image.jpg
```
```

That lets someone verify:

- did the model or fallback find the sidewalk

- what was the confidence
- what heading bias was produced

11. How To Test A Model On Live Camera

If someone wants to test the perception stack live before autonomous driving, use the preview test file:

```
[test_camera_preview.py] (/Users/ramsabavat/Desktop/rc_car_code/code/test_files/test_camera_preview.py)
```

This is the safest "see what the model is doing" test.

It shows:

- current camera livestream
- overlayed annotations
- method used
- confidence
- heading bias
- left/right edge status
- frame age

Typical run command:

```
```bash
python3 /Users/ramsabavat/Desktop/rc_car_code/code/test_files/test_camera_preview.py
```
```

This is important because it lets someone answer:

- is the model seeing the sidewalk
- is inference too slow
- are masks stable
- does the corridor center make sense

before letting the car drive itself

12. How To Decide Which Checkpoint Is Best

A good deployment decision should use both:

- offline metrics
- real-world image tests

Suggested ranking logic:

1. reject obviously weak checkpoints
2. compare mask mAP and box mAP
3. compare performance on unseen test images
4. compare performance on live or recent vehicle photos
5. inspect actual mask quality, not just hit counts

In this project, a model with the best internal metric was not always the best model for real deployment.

That is normal.

The best deployment model is the one that behaves best in the real use case.

13. How To Improve The Model

If masks exist but are inaccurate, the best improvements are usually:

- better labels

- more diverse hard examples
- cleaner annotation rules
- a better warm-start checkpoint
- slightly lower learning rate for refinement

Not the first thing to try:

- huge batch changes
- random augmentation changes
- many settings at once
- architecture hopping

High-value data to add:

- difficult turns
- grass-sidewalk boundaries
- driveway transitions
- shadow-heavy scenes
- broken sidewalks
- different times of day

14. Move The Best Checkpoint Into The Controller

Once you choose the best model, copy its `best.pt` into the controller app.

In this repo, that location is:

```
[best.pt] (/Users/ramsabavat/Desktop/rc_car_code/code/controller/current/rc_car_app/best.pt)
```

Then the controller-side vision code can load it directly.

That is the bridge between:

- model experimentation
- actual live vehicle behavior

15. Test In Stages Before Driving

Never go from "finished training" straight to full autonomous driving.

Use this order:

1. offline image tests
2. saved output folder review
3. live preview test window
4. stationary vehicle testing
5. very low-speed supervised driving
6. larger supervised autonomous tests

This is how you catch:

- latency problems
- wrong steering sign
- unstable masks
- poor confidence behavior

before damaging the vehicle or making a bad control decision

16. What Someone Else Should Copy From This Workflow

If another person wants to build their own model the same way, they should do this:

1. Mount the real camera they will use on the robot or vehicle.
2. Capture their own images from that actual setup.
3. Upload those images into Roboflow.
4. Label segmentation masks carefully in Roboflow.
5. Export as COCO segmentation.
6. Put the export into the project.
7. Start training from a YOLO segmentation checkpoint like `yolo26s-seg.pt`.
8. Save every run and compare `best.pt` checkpoints.
9. Test checkpoints on unseen images, not just validation metrics.
10. Use a live preview tool before trying real autonomous control.
11. Copy the chosen `best.pt` into the live controller code.
12. Test slowly and with supervision.

17. Concrete Example Workflow

A complete example for someone following this project style:

Data

- take RC car photos
- keep hard scenes
- use 1080p if possible

Labeling

- upload to Roboflow
- label `sidewalk`
- review labels carefully

Export

- export COCO segmentation from Roboflow
- place it in `code/ai\_models/...coco-segmentation`

Training

- use
[train_manual_sidewalk_model.py] (/Users/ramsabavat/Desktop/rc_car_code/code/ai_models/train_manual_sidewalk_model.py)
- start from `yolo26s-seg.pt`
- keep a clean baseline
- change one thing at a time

Evaluation

- compare `results.csv`
- compare `best.pt`
- run on real route images
- inspect saved prediction outputs

Local testing

- run
[sidewalk_image_test.py] (/Users/ramsabavat/Desktop/rc_car_code/code/test_files/sidewalk_image_test.py)
on single images
- run
[test_camera_preview.py] (/Users/ramsabavat/Desktop/rc_car_code/code/test_files/test_camera_preview.py)
for live preview

Deployment

- copy the chosen checkpoint into
[best.pt] (/Users/ramsabavat/Desktop/rc_car_code/code/controller/current/rc_car_app/best.pt)

- test the controller stack at low speed first

18. Final Advice

If someone wants to make their own image model and test it themselves, the most important lessons from this workflow are:

- collect real data from the actual camera
- label in Roboflow
- export cleanly
- start from `yolo26\*.pt`
- keep experiments controlled
- compare on real images, not just CSV metrics
- use a preview test before live driving
- deploy only the checkpoint that actually behaves best on unseen scenes

That is the practical end-to-end path from "I have a camera" to "I have a model I can actually test on my own system."